



MASTER 2 COMPUPhys - FREE PROJECT

2D Airfoil Simulation

Ali Fadlallah

Contents

1	Introduction	2
2	Physical model	2
2.1	NACA Airfoil	2
2.2	Fluid Dynamics in 2D Airfoil Simulation:	2
2.3	Boundary Conditions	4
2.4	Solution Proccedure	5
3	Functional Requirements of the Program	5
3.1	Description of Built Modules	5
3.2	Description of Built Object Types/Classes	6
4	Results	7
4.1	Insights and Analysis	7
4.2	Future Work	8
5	Conclusion	9

1 Introduction

This report provides insights into a Computational Fluid Dynamics (CFD) simulation focused on 2D airfoil dynamics. Developed in Python, the `AirfoilSolver` program aims to analyze airflow characteristics, leveraging numerical techniques and boundary conditions.

This report offers an overview of the simulation's functionality, input parameters, and expected outputs. Let's explore the mechanics of simulating airfoil behavior.

2 Physical model

2.1 NACA Airfoil

The simulation is based on the computational modeling of a 2D NACA (National Advisory Committee for Aeronautics) airfoil. NACA airfoils are widely used in aerodynamics and represent a standardized profile characterized by specific geometric parameters. The NACA airfoil employed in this simulation is defined by a four-digit code, with each digit conveying information about the airfoil's camber, thickness, and other aerodynamic features.

The chosen NACA airfoil influences the aerodynamic behavior of the simulation. We can load different NACA airfoil shapes using the `airfoils.py` package.

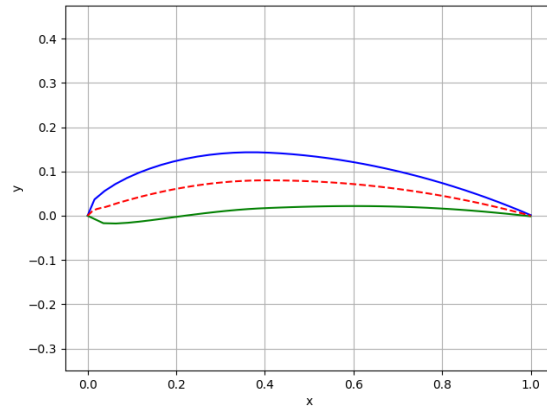


Figure 1: NACA 4813 shape

2.2 Fluid Dynamics in 2D Airfoil Simulation:

The program simulates the flow of air over a 2D airfoil using Computational Fluid Dynamics (CFD) principles. The fluid flow is governed by the Navier-Stokes equations, which are a set of partial differential equations describing the conservation of momentum for incompressible fluids. The simulation domain is discretized into a grid, and the evolution of velocity fields is numerically calculated over time.

Navier-Stokes Equations: Recall the Navier-Stokes equations for an incompressible fluid, where $\vec{v}$ is the velocity field:

$$\frac{\partial \vec{v}}{\partial t} + (\vec{v} \cdot \nabla) \vec{v} = -\frac{1}{\rho} \nabla p + \nu \nabla^2 \vec{v}$$

where u and v are the velocity components in the x and y directions, p is the pressure, ρ is the fluid density, ν is the kinematic viscosity, and ∇ represents the gradient operator.

Simplified Poisson Equation: The pressure field is determined by solving a Poisson equation:

$$\frac{\partial^2 p}{\partial x^2} + \frac{\partial^2 p}{\partial y^2} = b$$

Here is the system of differential equations: two equations for the velocity components u, v and one equation for pressure:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} = -\frac{1}{\rho} \frac{\partial p}{\partial x} + \nu \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right)$$

$$\frac{\partial v}{\partial t} + u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} = -\frac{1}{\rho} \frac{\partial p}{\partial y} + \nu \left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} \right)$$

$$\frac{\partial^2 p}{\partial x^2} + \frac{\partial^2 p}{\partial y^2} = -\rho \left(\frac{\partial u}{\partial x} \frac{\partial u}{\partial x} + 2 \frac{\partial u}{\partial y} \frac{\partial v}{\partial x} + \frac{\partial v}{\partial y} \frac{\partial v}{\partial y} \right)$$

Discretization:

The above equations can be discretized as follow:

We can discretize the pressure equation using 2nd-order central differences as follows:

$$\begin{aligned} & \frac{p_{i+1,j}^n - 2p_{i,j}^n + p_{i-1,j}^n}{\Delta x^2} + \frac{p_{i,j+1}^n - 2p_{i,j}^n + p_{i,j-1}^n}{\Delta y^2} \\ &= \rho \left[\frac{1}{\Delta t} \left(\frac{u_{i+1,j} - u_{i-1,j}}{2\Delta x} + \frac{v_{i,j+1} - v_{i,j-1}}{2\Delta y} \right) \right. \\ & \quad - \frac{u_{i+1,j} - u_{i-1,j}}{2\Delta x} \frac{u_{i+1,j} - u_{i-1,j}}{2\Delta x} \\ & \quad - 2 \frac{u_{i,j+1} - u_{i,j-1}}{2\Delta y} \frac{v_{i+1,j} - v_{i-1,j}}{2\Delta x} \\ & \quad \left. - \frac{v_{i,j+1} - v_{i,j-1}}{2\Delta y} \frac{v_{i,j+1} - v_{i,j-1}}{2\Delta y} \right] \end{aligned}$$

We use finite difference methods to discretize the u and v components of the velocity field as follows:

$$\begin{aligned} & \frac{u_{i,j}^{n+1} - u_{i,j}^n}{\Delta t} + u_{i,j}^n \frac{u_{i,j}^n - u_{i-1,j}^n}{\Delta x} + v_{i,j}^n \frac{u_{i,j}^n - u_{i,j-1}^n}{\Delta y} = \\ & - \frac{1}{\rho} \frac{p_{i+1,j}^n - p_{i-1,j}^n}{2\Delta x} + \nu \left(\frac{u_{i+1,j}^n - 2u_{i,j}^n + u_{i-1,j}^n}{\Delta x^2} + \frac{u_{i,j+1}^n - 2u_{i,j}^n + u_{i,j-1}^n}{\Delta y^2} \right) \end{aligned}$$

$$\begin{aligned} & \frac{v_{i,j}^{n+1} - v_{i,j}^n}{\Delta t} + u_{i,j}^n \frac{v_{i,j}^n - v_{i-1,j}^n}{\Delta x} + v_{i,j}^n \frac{v_{i,j}^n - v_{i,j-1}^n}{\Delta y} = \\ & - \frac{1}{\rho} \frac{p_{i,j+1}^n - p_{i,j-1}^n}{2\Delta y} + \nu \left(\frac{v_{i+1,j}^n - 2v_{i,j}^n + v_{i-1,j}^n}{\Delta x^2} + \frac{v_{i,j+1}^n - 2v_{i,j}^n + v_{i,j-1}^n}{\Delta y^2} \right) \end{aligned}$$

Rearranging, the momentum equation in the u direction becomes:

$$\begin{aligned} u_{i,j}^{n+1} = & u_{i,j}^n - u_{i,j}^n \frac{\Delta t}{\Delta x} (u_{i,j}^n - u_{i-1,j}^n) - v_{i,j}^n \frac{\Delta t}{\Delta y} (u_{i,j}^n - u_{i,j-1}^n) \\ & - \frac{\Delta t}{\rho 2 \Delta x} (p_{i+1,j}^n - p_{i-1,j}^n) \\ & + \nu \left(\frac{\Delta t}{\Delta x^2} (u_{i+1,j}^n - 2u_{i,j}^n + u_{i-1,j}^n) + \frac{\Delta t}{\Delta y^2} (u_{i,j+1}^n - 2u_{i,j}^n + u_{i,j-1}^n) \right) \end{aligned}$$

And the momentum equation in the v direction:

$$\begin{aligned} v_{i,j}^{n+1} = & v_{i,j}^n - u_{i,j}^n \frac{\Delta t}{\Delta x} (v_{i,j}^n - v_{i-1,j}^n) - v_{i,j}^n \frac{\Delta t}{\Delta y} (v_{i,j}^n - v_{i,j-1}^n) \\ & - \frac{\Delta t}{\rho 2 \Delta y} (p_{i,j+1}^n - p_{i,j-1}^n) \\ & + \nu \left(\frac{\Delta t}{\Delta x^2} (v_{i+1,j}^n - 2v_{i,j}^n + v_{i-1,j}^n) + \frac{\Delta t}{\Delta y^2} (v_{i,j+1}^n - 2v_{i,j}^n + v_{i,j-1}^n) \right) \end{aligned}$$

And finally the pressure p becomes:

$$\begin{aligned} p_{i,j}^n = & \frac{(p_{i+1,j}^n + p_{i-1,j}^n) \Delta y^2 + (p_{i,j+1}^n + p_{i,j-1}^n) \Delta x^2}{2(\Delta x^2 + \Delta y^2)} \\ & - \frac{\rho \Delta x^2 \Delta y^2}{2(\Delta x^2 + \Delta y^2)} \\ & \times \left[\frac{1}{\Delta t} \left(\frac{u_{i+1,j} - u_{i-1,j}}{2\Delta x} + \frac{v_{i,j+1} - v_{i,j-1}}{2\Delta y} \right) - \frac{u_{i+1,j} - u_{i-1,j}}{2\Delta x} \frac{u_{i+1,j} - u_{i-1,j}}{2\Delta x} \right. \\ & \left. - 2 \frac{u_{i,j+1} - u_{i,j-1}}{2\Delta y} \frac{v_{i+1,j} - v_{i-1,j}}{2\Delta x} - \frac{v_{i,j+1} - v_{i,j-1}}{2\Delta y} \frac{v_{i,j+1} - v_{i,j-1}}{2\Delta y} \right] \end{aligned}$$

These equations will be solved iteratively to obtain the velocity field and pressure distribution in the simulation domain.

2.3 Boundary Conditions

The implemented boundary conditions in the `AirfoilSolver` program ensure realistic and physically consistent interactions between the fluid and the simulation domain. The following boundary conditions are applied:

Velocity Boundary Conditions

- **Left Boundary:** The velocities on the left boundary are set to the freestream values $(u_{\text{freestream}}, v_{\text{freestream}})$.
- **Top, Bottom and Right Boundaries:** The velocities on the right boundary are set to the values of the neighboring grid points.
- **Airfoil Surface:** Velocities on the airfoil surface are set to zero, enforcing a no-slip condition.

Pressure Boundary Conditions

- **Left, Right, Top and Bottom Boundaries:** Are all set to a constant value (P_0) which is $P_0 = 0.5 * \rho * u_{\text{freestream}}$.

Additional Considerations

- **Airfoil Influence (Pending):** Currently, the influence of the airfoil on the flow is not fully implemented, it's defined in the `airfoil_influence()` method, but further adjustments are needed to accurately represent the airfoil's impact on pressure distribution. Thus, we omit it in the simulation.

2.4 Solution Procedure

Pressure, we iteratively solve the Poisson equation for pressure, as described above. Starting with an initial guess, the values $p_{i,j}$ are updated using the neighboring values of p , u and v at $(i+1,j)$ and $(i,j+1)$. In other words, we repeatedly apply the Poisson equation until the pressure reaches a quasi-steady state.

Velocities, Once the pressure field reaches its quasi-steady state via the Poisson equation, we use that field for the current time step, p_n , to solve for the velocity components u and v at the next timestep, $n+1$. Then rinse and repeat.

3 Functional Requirements of the Program

The primary goal of the program is to simulate the flow of air over a 2D airfoil using Computational Fluid Dynamics (CFD) techniques in Python. The program analyzes the aerodynamic characteristics, velocity and pressure distribution, and showcase them.

3.1 Description of Built Modules

The program consists of the following modules:

- `airfoil_solver.py`: Main module containing the `AirfoilSolver` class and related functions.
- `config.yaml`: Configuration file for input parameters.
- `main.py`: Main script to run the program and visualize results.

`config.yaml` - Input Data:

The file `config.yaml` contains all the input parameters necessary for the system:

- **`nx (int)`:** Number of grid points in the x-direction (even and less than 100).
- **`ny (int)`:** Number of grid points in the y-direction ($=nx$).
- **`length (float)`:** Length of the simulation domain.
- **`width (float)`:** Width of the simulation domain.
- **`time_step (float)`:** Time step for the simulation (~ 0.001).
- **`num_steps (int)`:** Number of time steps for the simulation.

- **nit (int):** Number of iterations for the pressure solver.
- **u_freestream (float):** Initial velocity in the x-direction.
- **v_freestream (float):** Initial velocity in the y-direction.
- **density (float):** Density of the fluid.
- **viscosity (float):** Viscosity of the fluid.
- **airfoil_code (string):** Code specifying the type of airfoil (e.g., "4813").

3.2 Description of Built Object Types/Classes

AirfoilSolver Class

- **Attributes:**
 - nx, ny, length, width, dt, num_steps, nit, u_freestream, v_freestream, rho, nu, airfoil_code
- **Functional Requirements of Methods:**
 - `__init__`: Initializes the object and sets up simulation parameters.
 - `initialize_flow_field`: Initializes the flow field variables.
 - `load_airfoil_coordinates`: Loads airfoil coordinates and scales them to the simulation domain.
 - `apply_boundary_velocity`: Applies boundary conditions for velocity.
 - `apply_boundary_pressure`: Applies boundary conditions for pressure.
 - `build_up_b`: Builds up the `b` array used in pressure computation.
 - `calculate_pressure`: Computes pressure using a pressure Poisson equation.
 - `calculate_u_velocity`, `calculate_v_velocity`: Updates x and y-velocities using explicit finite difference method.
 - `airfoil_flow`: Executes the simulation by iterating over time steps.
 - `plot_airfoil`: Plots the airfoil.
 - `visualize`: Visualizes simulation results using contour plots and streamlines.

Possible Outputs

- Velocity field (`u` and `v` arrays).
- Pressure field (`p` array).
- Visualization outputs:
 - Velocity field visualizations.
 - Pressure contours.

Exceptions: The program will stop without providing data if NaN or Infinite values are detected in the velocity field. This usually indicates issues with configuration parameters, most probably too small or big mesh size, and/or small `time_step`.

Running time: Average running time depends on the complexity of the simulation. For the provided configuration, it takes less than a minute on average.

4 Results

The simulation results are visually presented in Figure 2, showcasing the airflow characteristics, pressure distribution, and velocity field around the 2D NACA airfoil in reduced units.

The plot illustrates the geometric configuration of the NACA airfoil, distinguishing between its upper (blue) and lower (green) surfaces. Contour plots vividly showcase the pressure distribution across the simulation domain, complemented by streamlines conveying the velocity field of the airflow.

4.1 Insights and Analysis

There is a high pressure region right in front of the airfoil, then it decrease gradually as we pass over the airfoil, but the pressure around the lower surface appear slightly higher than its counterpart around the upper surface.

This is accompanied by higher velocities (longer streamlines) on the upper surface compared to the lower surface align with physical expectations. This suggests that our approach, although a simplified model, produces somewhat a reasonable results in a first exposure to fluid dynamics.

However, it's worth noting that this method has limitations. The approach involves gathering data points from the airfoil package (in an attempt to make the program usable with any airfoil shapr from NACA, and it does!), fitting a curve to generate upper and lower surfaces, and determining indices inside the shape for boundary conditions. This might not be optimal for smaller domains due to a limited number of points in the shape. Additionally, the obtained shape may lack smoothness. The influence of the airfoil on the flow didn't produce any resonable results so it was omitted from the shown simulation

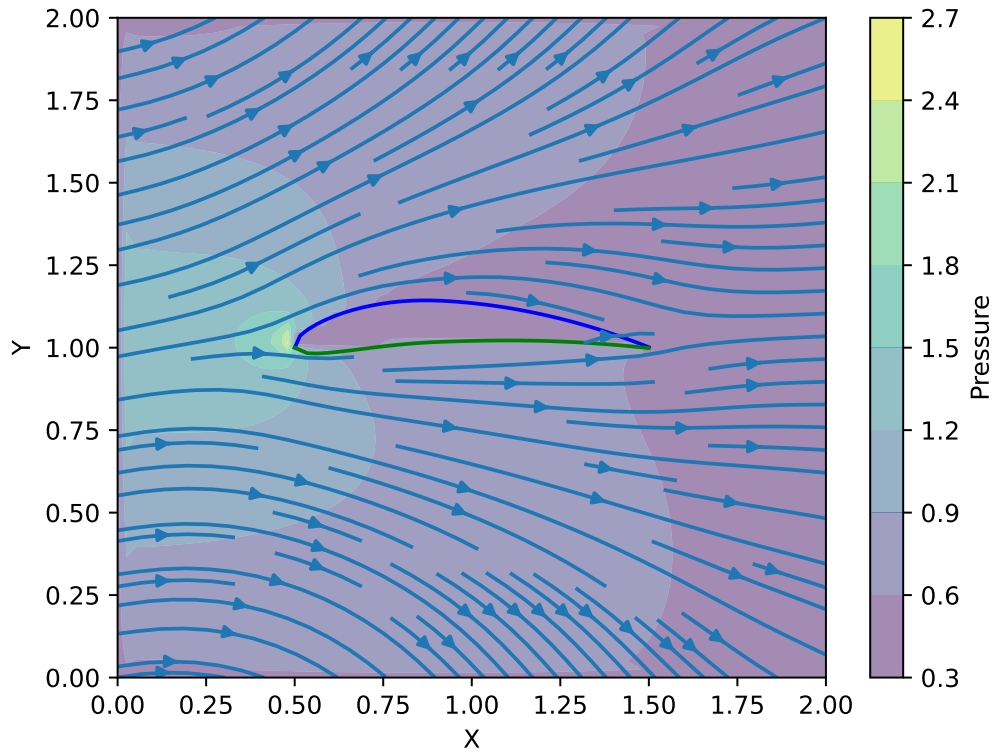


Figure 2: Simulation Results for NACA Airfoil

Simulation Parameters:

- nx: 80, ny: 80
- length: 2.0, width: 2.0
- time_step: 0.001
- num_steps: 700
- nit: 50
- u_freestream: 1.0, v_freestream: 0.2
- density: 1.0, viscosity: 0.1
- airfoil_code: "4813"

4.2 Future Work

Refining the computation of the airfoil surface can include using analytical equations for generating points, which may offer a more accurate representation. And exploring advanced boundary conditions and turbulence models could lead to a more realistic simulation.

5 Conclusion

In conclusion, the current model has provided reasonable results, offering a foundational understanding of airflow over a 2D NACA airfoil. While the simulation outcomes are satisfactory for an initial exploration, there are evident areas for improvement.

Future iterations should focus on enhancing accuracy by implementing analytical equations for airfoil surface computation, and exploring advanced boundary conditions and turbulence..

This project, serving as an opportunity to apply Object-Oriented Programming principles in the context of fluid dynamics..